TrueTracts-Style Market Analysis Implementation

Overview

Transform the Market Analysis tab to match TrueTracts architecture: add GAM (Generalized Additive Model) statistical analysis, interactive map with polygon drawing, locational value heatmap, and multi-method adjustment framework.

Core Components

1. Python GAM Microservice

Create a Python-based statistical analysis service to handle GAM computations:

New Files:

- `server/python/gam_service.py` - Flask/FastAPI microservice exposing GAM endpoints
- `server/python/requirements.txt` - Python dependencies (pyGAM, numpy, pandas, scipy)
- `server/python/spatial_analysis.py` - GAM spatial component calculator for heatmap
- `server/python/time_adjustment.py` - Non-linear time adjustment via GAM

Key Implementation:

- GAM model: `log(price) ~ s(time) + s(lat, lng) + s(gla) + s(quality) + s(condition)`
- Spatial smooth function `s(lat, lng)` produces the locational value layer
- API endpoints: `/gam/fit`, `/gam/predict`, `/gam/heatmap`

2. Database Schema Updates

Add PostGIS support for spatial queries:

Files to Modify:

- `shared/schema.ts` - Add GAM results types, heatmap data structures
- Run SQL migration to enable PostGIS extension

New Types:

```typescript
export interface GamModelResult {
  orderId: string;
  coefficients: Record<string, number>;
  smoothTerms: { time: number[][]; spatial: number[][]; };
  rsquared: number;
  aic: number;
  residuals: number[];
}

export interface HeatmapPoint {
  lat: number;
  lng: number;
  locationValue: number; // adjustment in $ or %
  confidence: number;
}
```

3. Interactive Map Implementation

Enhance `client/src/components/map/MarketMap.tsx` with full Leaflet integration:

Changes:

- Replace placeholder with actual Leaflet map using `react-leaflet`
- Implement polygon drawing/editing with `leaflet-draw`
- Add subject marker (red) and comp markers (blue)
- Real-time filtering: as user draws polygon, filter comps with PostGIS `ST_Contains`
- Display heatmap layer with color gradient (red = premium, blue = discount)

Key Libraries (already installed):

- `leaflet` for base map
- `leaflet-draw` for polygon tools
- `@turf/turf` for client-side geospatial calculations

4. Market Analysis API Routes

Add new endpoints in `server/routes.ts`:

New Endpoints:

- `POST /api/orders/:id/market/gam/compute` - Trigger GAM analysis
- `GET /api/orders/:id/market/gam/results` - Get GAM model results
- `GET /api/orders/:id/market/heatmap` - Get heatmap grid data
- `POST /api/orders/:id/market/polygon` - Save user-drawn polygon
- `POST /api/orders/:id/market/filter` - Filter records by polygon (using PostGIS)

5. Enhanced Market Analysis UI

Update `client/src/pages/orders/[orderId]/market.tsx`:

New Tabs/Sections:

- Add "Locational Analysis" tab showing heatmap and spatial insights
- Update "Time Adjustments" tab to show GAM non-linear curve vs. linear comparison
- Add "Multi-Method Adjustments" section showing:
  - GAM-based adjustment
  - Depreciated Cost baseline
  - Peer adjustments (placeholder for now)
  - Sensitivity analysis

Visualizations:

- Non-linear time trend chart (GAM smooth curve)
- Comparison chart: GAM vs. Theil-Sen vs. OLS
- Heatmap legend with $ value scale
- Market area polygon with color-coded zones

6. Python Service Integration

Add Node.js → Python bridge in `server/`:

New Files:

- `server/lib/gam-client.ts` - HTTP client to call Python GAM service
- Add Python service startup to development workflow

Integration Pattern:

```typescript
// Example usage
const gamResults = await callGamService('/gam/fit', {
  records: marketRecords,
  subjectLat: 30.267,
  subjectLng: -97.743
});
```

Implementation Priority

1. **Python GAM Service** - Core statistical engine (highest priority)
2. **Database Schema** - Add GAM result storage and PostGIS
3. **Interactive Map** - Polygon drawing and real-time filtering
4. **API Routes** - Connect front-end to GAM service
5. **Heatmap Layer** - Visualize spatial GAM component
6. **UI Enhancements** - Multi-method display and improved charts

Files to Create

- `server/python/gam_service.py`
- `server/python/requirements.txt`
- `server/python/spatial_analysis.py`
- `server/python/time_adjustment.py`
- `server/lib/gam-client.ts`
- `migrations/add_postgis.sql`

Files to Modify

- `client/src/components/map/MarketMap.tsx` (replace placeholder)

```
- `client/src/pages/orders/[orderId]/market.tsx` (add GAM features)
- `shared/schema.ts` (add GAM types)
- `server/routes.ts` (add GAM endpoints)
- `package.json` (add react-leaflet if needed)

## Technical Notes

- GAM computations are CPU-intensive; run Python service on separate
process/container
- Heatmap should use grid interpolation (e.g., 50x50 grid over market
area)
- Cache GAM results in database to avoid re-computation
- PostGIS spatial index on lat/lng for fast polygon filtering
- Use `pyGAM` library for GAM implementation (well-documented, stable)
```

8 To-dos • Done sequentially

+ New

- Create Python GAM microservice with Flask/FastAPI, pyGAM model, and API endpoints for fit/predict/heatmap
- Add GamModelResult and HeatmapPoint types to shared/schema.ts
- Create PostGIS migration and update database configuration
- Create server/lib/gam-client.ts HTTP client to communicate with Python service
- Add GAM-related API endpoints in server/routes.ts
- Replace MarketMap placeholder with full Leaflet implementation: polygon drawing, markers, real-time filtering
- Add heatmap visualization layer to MarketMap using GAM spatial component data
- Update market.tsx: add Locational Analysis tab, GAM vs linear comparison charts, multi-method adjustments display